UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
Programação Avançada — 2026.2
Prof. Dr. Hertz Wilton de Castro Lins

Mochila Booleana (Knapsack 0/1)

Parte I — Implementação do algoritmo

Equipe: Victor Manuel Valentim Cordeiro
Algoritmo: nº 5 — Mochila Booleana (Knapsack 0/1)
GitHub: https://github.com/alphachief13/sigaa-simulado
Natal/RN — outubro de 2026

Mochila Booleana (Knapsack 0/1) — Parte I

Algoritmo nº 5 da lista: **Programação Dinâmica**. Implementação: `sigaa_sim/algoritmos/mochila.py`. Testes: `tests/test_mochila.py`. Benchmark: `scripts/benchmark_mochila.py`.

1. Características e aplicações

Problema. São dados `n` itens, cada um com peso inteiro $p_i \geq 0$ e valor $v_i \geq 0$, e uma capacidade `W`. O objetivo é escolher um subconjunto `S` que maximize $\Sigma\, v_i$ com $\Sigma\, p_i \leq W$. Cada item entra **no máximo uma vez**: ou é levado inteiro, ou fica de fora (daí "0/1", "booleana").

Por que Programação Dinâmica. O problema tem:

- **Subestrutura ótima.** A melhor solução com os `i` primeiros itens e capacidade `w` ou ignora o item `i`, ou leva o item `i` e resolve otimamente o resto com $w - p_i$.
- **Subproblemas sobrepostos.** Os mesmos pares `(i, w)` aparecem em muitos ramos da recursão. Guardá-los numa tabela evita recalcular.

Recorrência, com `T[i][w]` = melhor valor usando os `i` primeiros itens e capacidade `w`:

```
T[0][w] = 0
T[i][w] = T[i−1][w]                                   se p_i > w
T[i][w] = max( T[i−1][w],  T[i−1][w − p_i] + v_i )    caso contrário
resposta = T[n][W]
```

Complexidade. Tempo Θ(n·W) e memória Θ(n·W). O tempo é **pseudo-polinomial**: depende do valor numérico de `W`, não do tamanho da entrada em bits. O problema de decisão é NP-completo, mas fica viável quando `W` é pequeno. A versão `valor_maximo` usa um vetor só, com memória Θ(W), quando basta o valor ótimo.

Por que não o guloso. Ordenar por densidade `v/p` é ótimo na mochila *fracionária*, mas não na 0/1. Contraexemplo com W = 7:

item	peso	valor	v/p
A	1	1	1,00
B	3	4	1,33
C	4	5	1,25
D	5	7	1,40

O guloso leva D e depois A (valor 8). O ótimo é B + C (valor 9). O guloso perde 11,1%.

Aplicações. Alocação de orçamento, seleção de investimentos e de projetos, carregamento de cargas, escolha de anúncios por espaço, corte de materiais e alocação de tempo.

Aplicação neste projeto. O agente monta o **plano de estudo do dia**:

Mochila	Plano de estudo
capacidade `W`	minutos livres no dia ÷ 5 (ex.: 90 min → 18 blocos)
item	sessão de estudo candidata para um compromisso (prova/tarefa)
peso p_i	duração da sessão em blocos de 5 min (35 min → 7; revisão curta de 15 min → 3)
valor v_i	`1000 × prioridade`, com retorno decrescente: k-ésima sessão do mesmo compromisso × $0{,}6^k$; revisão curta × 0,3

A prioridade vem da fórmula explicável do planejador: `6/(1+dias) + 1,5·peso/10 + 0,5·frequência/5 + 1,0·pendência`. A Mochila escolhe o conjunto de sessões de **maior valor total que cabe no tempo do dia**.

2. Entrada de dados

1. **Origem.** O agente sincroniza com o **SIGAA Simulado** (**Coletar e processar** no painel): tarefas (prazo com hora), avaliações do plano de curso (data, peso e escopo), materiais e progresso.
2. **Compromissos.** `collect_commitments` filtra tarefas e avaliações com prazo nos próximos 21 dias e calcula a prioridade de cada uma.
3. **Itens.** `planner.candidatas` gera, para cada compromisso, até `⌊minutos/35⌋` sessões longas e uma revisão curta, como `ItemMochila(nome, peso, valor, ref)`.
4. **Capacidade.** `[planner].availability_minutes` por dia da semana (ou `--minutos`).
5. **Validação da entrada.** Pesos, valores e capacidade precisam ser **inteiros ≥ 0**. Caso contrário, `ValueError` com o item e o campo. Isso rejeita `bool`, `float` e negativos.

Entrada didática fixa: botão **Exemplo didático** no Controle (CLI: `sigaa-sim mochila --exemplo`).

3. Como foi implementado

Estruturas de dados.

- `ItemMochila` (dataclass imutável): `nome`, `peso`, `valor` e `ref` (objeto de domínio, fora da comparação).
- Tabela `T`: lista de `n+1` listas com `W+1` inteiros. A linha 0 representa "nenhum item".
- `ResultadoMochila`: `valor_total`, `peso_total`, `capacidade`, `indices`, `escolhidos` e `tabela` (opcional, para exibição).

Fluxo de `mochila_01(itens, W)`.

```
validar entrada
T ← matriz (n+1)×(W+1) de zeros
para i = 1..n:                        # cada item
    para w = 0..W:                    # cada capacidade
        T[i][w] ← T[i-1][w]           # não leva
        se p_i ≤ w e T[i-1][w-p_i]+v_i > T[i][w]:
            T[i][w] ← T[i-1][w-p_i]+v_i   # leva (só se for estritamente melhor)
# reconstrução, de trás para frente
w ← W
para i = n..1:
    se T[i][w] ≠ T[i-1][w]: item i foi escolhido; w ← w − p_i
```

A comparação estrita (`>`) faz os empates preferirem *não* levar o item. Isso torna o resultado determinístico e favorece os índices menores.

Exemplo da tabela (**Exemplo didático** com *tabela*; CLI: `sigaa-sim mochila --exemplo --tabela`; `*` = escolhido):

```
             w=0   1   2   3   4   5   6   7
  ∅             0   0   0   0   0   0   0   0
  A (p=1,v=1)   0   1   1   1   1   1   1   1
 *B (p=3,v=4)   0   1   1   4   5   5   5   5
 *C (p=4,v=5)   0   1   1   4   5   6   6   9
  D (p=5,v=7)   0   1   1   4   5   7   8   9
```

`T[4][7] = T[3][7] = 9`, então D não entra. Em `T[3][7] ≠ T[2][7]`, C entra (w = 3). Em `T[2][3] ≠ T[1][3]`, B entra (w = 0).

Variante com memória Θ(W) (`valor_maximo`): usa um vetor `V[0..W]` percorrido com `w` **decrescente**. Assim, `V[w-p]` ainda é o valor da linha anterior e cada item é usado uma vez só. Com `w` crescente, viraria a mochila *ilimitada*.

Integração. `planner.selecionar_sessoes` chama `mochila_01`, ordena as sessões escolhidas por prioridade e prazo, e `fallback_tasks` transforma cada uma numa tarefa com passos e materiais. O plano registra a nota "sessões escolhidas pela Mochila 0/1: k de n candidatas, x/y min, valor v".

4. Como foi validado

`pytest tests/test_mochila.py`: **319 casos**, todos passando.

Verificação	Como
Corretude contra a referência	300 instâncias aleatórias (n ≤ 12, W ≤ 40, sementes 0–299) comparadas com a **força bruta** (todos os 2^n subconjuntos)
Reconstrução consistente	os itens escolhidos somam exatamente o valor ótimo, cabem na capacidade e não se repetem
Variante Θ(W)	`valor_maximo == mochila_01.valor_total` em todas as instâncias
Guloso	sempre viável e ≤ ótimo; o contraexemplo A/B/C/D mostra o guloso perdendo
Casos clássicos	instância de livro-texto (60/100/120, W = 50 → 220)
Bordas	sem itens, W = 0, nenhum item cabe, peso zero, valor zero, encaixe exato
Entradas inválidas	capacidade negativa, `float`, `bool`, peso ou valor negativo → `ValueError`
0/1 de fato	o item (1,10) não é usado 3 vezes com W = 3
Propriedades	valor ótimo monótono em W; determinismo em empates
Integração	`tests/test_simulador.py`: o plano gerado sobre o SIGAA Simulado respeita o tempo para 15, 50, 90 e 180 min

Desempenho empírico (`scripts/benchmark_mochila.py`, mediana de 3 execuções, Python 3.12):

n	W	n·W	tempo (ms)	ns/célula
25	250	6.250	0,5	74
50	500	25.000	2,2	89
100	1000	100.000	10,9	109
200	2000	400.000	44,3	111

O custo por célula fica quase constante (~65–120 ns), o que confirma Θ(n·W). A força bruta dobra a cada item: com n = 20 (≈10^6 subconjuntos), leva 761 ms contra 0,14 ms da PD.

No caso real do projeto (≈40 candidatas, W = 18), a tabela tem cerca de 760 células e é resolvida em menos de 1 ms.